Rigid-Body Ackermann Model: A Review

Lucas Libshutz
Cornell University, Mechanical and Aerospace Engineering

***Abstract*—For modern autonomous driving of high speed vehicles, high-accuracy localization is an essential requirement for proper control and planning, especially in competition environments. More specifically, for the Shell Eco Marathon, a car that is going 20+ MPH and on an angled track needs to have high confidence in both angle and position in order to control the car properly on a banked track. This work aims to provide a rigid body based EKF that can provide the orientation and position data to a high level of accuracy, while maintaining model fidelity and speed at runtime. Using JAX to formulate automatically differentiable dynamics, the EKF runs at *0.5 ms* for each predict + update iteration, and results in an average RMSE of approximately *2 cm* and *0.35 m/s* for position and velocity respectively.**

I. Goal

One of the main motivations for this project was to create a base on which the team could test and run an autonomous car in a low stakes environment, and use that to tune the kinematic parameters and the dynamic assumptions for the larger, more complex vehicle that would be tested afterwards. Because of the complex nature of the dynamics of the larger car, some of the main aspects of the system that were modeled included:

- Contact dynamics in z of the tires and the ground
- Orientation and tilt of the vehicle in roll, pitch, *and* yaw
- Tire slip parameters in both lateral and longitudinal directions

To do the last part is especially important for a larger car in a variable driving environment, where variable amounts of tire slip can drastically impact the localization accuracy of a filter, and contribute to large amounts of uncertainty.

II. Derivation of Relevant Dynamics

A. Notation

- $\mathcal{I}$ denotes an intertial frame, with its z axis pointing in the direction against gravity.
- The position vector in frame $\mathcal{A}$ to a point Q relative to the frame origin is denoted as ${}^{\mathcal{A}}\mathbf{p}_Q$.
- ${}^{\mathcal{A}}R_{\mathcal{B}}$ denotes the rotation matrix from frame $\mathcal{B}$ to frame $\mathcal{A}$. This is such that ${}^{\mathcal{A}}\mathbf{p} = {}^{\mathcal{A}}R_{\mathcal{B}}{}^{\mathcal{B}}\mathbf{p}$.
- Let $[\mathbf{x}]_\times \in \mathbb{R}^{3\times 3}$ be the skew-symmetric matrix such that $[\mathbf{x}]_\times \mathbf{y} = \mathbf{x} \times \mathbf{y}$, where $\times$ is the cross product operator in $\mathbb{R}^3$.

B. System Dynamics Formulation

With the above notation in mind, we formulate the state of the vehicle as a whole as the following vector:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \in \mathbb{R}^3 \\ R \in SO(3) \\ \mathbf{v} \in \mathbb{R}^3 \\ {}^{\mathcal{B}}\boldsymbol{\omega} \in \mathbb{R}^3 \\ \boldsymbol{\omega}_w \in \mathbb{R}^4 \end{bmatrix}$$

And the corresponding control input is also defined as:

$$\mathbf{u} = \begin{bmatrix} \delta \in \mathbb{R} \\ \boldsymbol{\tau} \in \mathbb{R}^2 \end{bmatrix}$$

From this, we now derive the full state space model of the system. First, we define the translational and rotional kinematics of the car as a whole via Newton's second law:

$$\begin{aligned} m\dot{\mathbf{v}} &= m\mathbf{g} + \sum_i \mathbf{f}_i \\ \dot{\mathbf{p}} &= \mathbf{v} \end{aligned} \tag{1}$$

Where $\mathbf{f}_i \in \mathbb{R}^3$ comes from the wheel dynmaics, detailed later. And similarly, for rotational kinematics:

$$\begin{aligned} \mathbb{I}_G{}^{\mathcal{B}}\dot{\boldsymbol{\omega}} + {}^{\mathcal{B}}\boldsymbol{\omega} \times \left(\mathbb{I}_G{}^{\mathcal{B}}\boldsymbol{\omega}\right) &= \boldsymbol{\tau}(\mathbf{x}, \mathbf{u}) \\ \dot{R} &= R[{}^{\mathcal{B}}\boldsymbol{\omega}]_\times \end{aligned} \tag{2}$$

And the formulation for the update of the orientation uses the built-in `jaxlie.SO3.exp` formulation, which evolves on $SO(3)$ while using a quaternion ($q \in S^3$) representation internally within the `jaxlie` library.

The quantity $\mathbf{f}_i$ is the force generated by the i-th wheel, and is calculated by both contact dynamics in z and slip dynamics in both x and y. To calulate the wheel dynamics, we first start wth the normal force $f_{z,i}$:

$$f_{z,i} = k_n \cdot \mathrm{ReLU}(z_0 - z) + c_n \cdot \mathrm{ReLU}(0 - v_z) \tag{3}$$

where $f_{z,i}$ is enforced to be positive, such that the car is not pulled into the ground (i.e. $z_0 - z > 0$). z_0 is the initial height, and z is the current height of the wheel. v_z is the vertical velocity of the wheel, and k_n and c_n are the stiffness and damping coefficients for the normal force, respectively.

Then, the wheel slip *ratios* are then found via the Pacejka slip model, which is a commonly used empirical model for tire slip dynamics. The slip ratios are defined as follows:

$$\begin{aligned} \kappa_i &= \kappa_{\max} \cdot \frac{r\omega_i - v_{t,i}}{\sqrt{v_{t,i}^2 + \epsilon^2}} \\ \alpha_i &= \arctan\left(\frac{v_{n,i}}{\sqrt{v_{t,i}^2 + \epsilon^2}}\right) \end{aligned} \tag{4}$$

In equation (4), the tolerance ϵ is added to prevent division by zero when the tangential velocity $v_{t,i}$ is very small. Furthermore, κ and α are also both bounded by a $\tanh$ function each, in order to bound large slip changes that may induce negative RPM values in the model. The slip ratios κ_i and α_i are then used to calculate the longitudinal and lateral forces generated by the wheel, via $f^\star_{x,i} = C_\kappa \cdot \kappa$ and $f^\star_{y,i} = -C_\alpha \cdot \alpha$, where C_κ and C_α are the longitudinal and lateral slip stiffness coefficients, respectively. From this, all forces are scaled by the maximum normal force, formulated as:

$$\begin{aligned} f_{\max} &= \mu \cdot f_{z,i} \\ \text{scale} &= \frac{f_{\max}}{\sqrt{\left(f^\star_{x,i}\right)^2 + \left(f^\star_{y,i}\right)^2 + \left(f_{\max}\right)^2}} \end{aligned} \tag{5}$$

From this, both $f^\star_{x,i}$ and $f^\star_{y,i}$ are scaled by the above quantity, such that the total force generated by the wheel is bounded by the maximum friction cone specified by μ. Finally, the forces are then rotated into the inertial frame and summed across all wheels to get the total force on the car, which is then used in the translational dynamics of the car as shown in equation (1).

The one other aspect of the dynamics that remains is the spin dynamics of each wheel specifically, which we define as a damped rotating disk as follows:

$$I_w \dot{\omega}_{w,i} = \tau_{\text{cmd},i} - r f_{x,i} - b_w \omega_{w,i} \tag{6}$$

Simiarlly, for the passive wheels on the car (i.e. front wheels for RWD), the dynamics are simplified to relax toward the standard kinematic rolling constraint:

$$\dot{\omega}_{w,i} = \frac{1}{\tau_f}\left(\frac{v_{t,i}}{r} - \omega_{w,i}\right) \tag{7}$$

The full state update blends the wheel rotation dynamics via a motor mask, such that $\dot{\omega}_w = \texttt{mask} \cdot \dot{\omega}_{\text{drive}} + (1 - \texttt{mask}) \cdot \dot{\omega}_{\text{passive}}$.

All combined together, the system of equations that evolve our state *cannot* be formed into a linear state-space model such as $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, due to the nonlinearities in the slip model and the rotation dynamics. However, the system can be linearized around a nominal trajectory, which is what is done in the next section.

III. Extended Kalman Filter Formulation

A. Error State Formulation

While most of the properties of the state $\mathbf{x}$ can be updated via the form $\mathbf{x} \leftarrow \mathbf{x} + K\nu$, we cannot do the same for the orientation R, since it evolves on the manifold $SO(3)$. If we ran a standard EKF with a quaternion directly in the state vector, the additive update would push q off of the unit sphere required to properly represent rotations.

In order to solve this problem, this work formulates a filter in **error state**. This means that instead of tracking full $SO(3)$, we instead formulate the error orientation as $\delta\boldsymbol{\theta} \in \mathbb{R}^3$, which lives in the tangent space $\mathfrak{so}(3) \cong \mathbb{R}^3$. Doing so allows us to implement a standard state update into the error state EKF, whose state is defined as:

$$\delta\mathbf{x} = \begin{bmatrix} \delta\mathbf{p} \in \mathbb{R}^3 \\ \delta\boldsymbol{\theta} \in \mathbb{R}^3 \\ \delta\mathbf{v} \in \mathbb{R}^3 \\ \delta{}^{\mathcal{B}}\boldsymbol{\omega} \in \mathbb{R}^3 \\ \delta\boldsymbol{\omega}_w \in \mathbb{R}^4 \end{bmatrix}$$

where all updates for non-orientation states are standard, except for $\delta\boldsymbol{\theta}$, which is used to update the orientation as follows:

$${}^{\mathcal{B}}R_{\mathcal{I}} + \leftarrow {}^{\mathcal{B}}R_{\mathcal{I}} \cdot \exp\left([\delta\boldsymbol{\theta}]_\times\right) \tag{8}$$

B. Filter Prediction

As the components of the system are nonlinear, as stated before we do not have a standard Kalman filter predict step; rather, we propagate the nominal state $\hat{\mathbf{x}}_{k+1|k}$ as:

$$\hat{\mathbf{x}}_{k+1|k} = f(\hat{\mathbf{x}}_{k|k}, \mathbf{u}_k) \tag{9}$$

where f is the integration step of the `CarModel` class, implemented in the library. Similarly for the covariance, we do not have a standard $P_{k+1|k} = FP_{k|k}F^T + Q$ update, but rather we linearize the dynamics around the nominal trajectory to get the Jacobian for the error dynamics, as:

$$F_k = \left.\frac{\partial f}{\partial \delta\mathbf{x}}\right|_{\hat{\mathbf{x}}_{k|k}, \mathbf{u}_k} \tag{10}$$

This Jacobian is calculated via the automatic diffentation feature in JAX. All classes are formulated as *pytrees*, which allow full XLA compilation of the program as a graph, and allow for efficient Jacobian calculation in both forward and reverse mode. This then feeds into the covariance propagation as:

$$P_{k+1|k} = F_k P_{k|k} F_k^T + Q \tag{11}$$

As $\delta\mathbf{x}$ is reset to zero after each error injection, the prediction error state mean will *always be zero*, as the predict step will only propagate covariance.

C. Filter Update and Sensor Models

We define the innovation in the filter $\boldsymbol{\nu} = \mathbf{z} - h(\hat{\mathbf{x}})$, with the measurement Jacobian of a sensor model h defined as:

$$H_k = \left.\frac{\partial h}{\partial \delta\mathbf{x}}\right|_{\hat{\mathbf{x}}_{k+1|k}} \tag{12}$$

Just as before, the Jacobian H_k at timestep k is calculated via JAX automatic differentiation. With the above, we then use this for three different sensors and their respective measurement models:

1) **GPS**: The GPS sensor model is defined as a simple position measurement in the inertial frame, with the measurement model defined as:

$$h_{\text{GPS}}(\mathbf{x}) = {}^{\mathcal{I}}\mathbf{p}_G \in \mathbb{R}^3 \tag{13}$$

2) **IMU**: The IMU sensor model is defined as the standard accelerometer and gyroscope model, which is used to measure (1) the angular velocity in x, y, and z, and is used to validate the acceleration against gravity ${}^{\mathcal{I}}\mathbf{a}_G$. This means that we have two measurement models from the IMU, formulated as:

$$\begin{aligned} h_{\text{gyro}}(\mathbf{x}) &= {}^{\mathcal{B}}\boldsymbol{\omega} \in \mathbb{R}^3 \\ h_{\text{acc}}(\mathbf{x}) &= {}^{\mathcal{B}}R_{\mathcal{I}}^{\top}({}^{\mathcal{I}}\mathbf{a}_G - \mathbf{g}) \in \mathbb{R}^3 \end{aligned} \tag{14}$$

3) **Wheel Encoders**: The wheel encoders are used to measure the angular velocity of each wheel, which is then used to validate the slip dynamics in the model. Due to current inconsistencies with the front wheel slip dynamics, we only use the rear wheel encoder measurements for the update step, which is formulated as:

$$h_{\text{wheels}}(\mathbf{x}) = \boldsymbol{\omega}_{w(\text{rear})} \in \mathbb{R}^2 \tag{15}$$

From the GPS, wheel encoders, and IMU, the update step via the innovation $\boldsymbol{\nu} = \mathbf{z} - h(\hat{\mathbf{x}})$ and observation Jacobian H_k batched across all measurements can be formulated as follows:

1) Define the innovation covariance:

$$S_k = H_k P_{k+1|k} H_k^T + R \tag{16}$$

2) Define the Kalman gain:

$$K_k = P_{k+1|k} H_k^T S_k^{-1} \tag{17}$$

3) Update the error state mean and covariance, using the *Joseph form*, which guarantees the covariance to be positive definite, even in the presence of numerical instability:

$$\begin{aligned} \delta\mathbf{x}_{k+1|k+1} &= K_k\boldsymbol{\nu} \\ P_{k+1|k+1} &= (I - K_k H_k)P_{k+1|k}(I - K_k H_k)^{\top} \\ &\quad + K_k R K_k^{\top} \end{aligned} \tag{18}$$

Finally, after each update step, we reset $\delta\hat{\mathbf{x}} \leftarrow \mathbf{0}$, such that we can keep the error state as a mathematical formality rather than another possibility for numerical instability.

IV. Results

To evaluate the model and the filter, we ran two separate trajectories to understand the validity of the dynamics and the filter:

1) **A straight line with left turn**, to ensure that the filter does not diverge when the system is not fully observable. In this case, the system is not fully observable because the steering angle is zero for the straight line portion of the trajectory, so the filter cannot estimate the steering angle during that time. However, once the turn begins, the filter should be able to estimate the steering angle and converge to the true state.
2) **A sinusoidal trajectory**, to ensure that the filter can track a more complex trajectory and that the model is accurate enough to capture the dynamics of the system. In this case, the system is fully observable because the steering angle is non-zero throughout the trajectory, so the filter should be able to estimate the state accurately and converge to the true state.

These tests were done first for a small RC mini car, assuming with the following initial parameters:

- Chassis dimensions of $26 \times 16 \times 6$ cm, and a mass of 1.5 kg.
- A ground fricton coefficient of $\mu = 0.9$
- Contact spring parameters $k_n = 2000$, $c_n = 50$
- Wheels with radius 3 cm, and mass 50 g each.

The contact spring parameters were chosen to result in a damping ratio as the following:

$$\zeta = \frac{c_n}{2\sqrt{k_n \cdot m_{\text{wheel(point)}}}} = 1.00416$$

The reason that we use $m_{\text{wheel(point)}}$ here instead of the wheel mass itself is because of the lack of suspension within this model: the dynamics formulated here treat the wheel as a rigid transform between the center of mass and the wheel axis of rotation, with no translational degree of freedom.

Along with the kinematic parameters mentioned above, both tests were run with the following sensor σ values, to simulate an inherently noisy system to evaluate how well the dynamics would drown out sensor noise:

- GPS: $\sigma = 1.5$ m
- Gyro: $\sigma = 0.01$ rad/s
- Accelerometer: $\sigma = 0.1$ m/s^2
- Wheels: $\sigma = 0.01$ rad/s

A. Straight & Left Turn

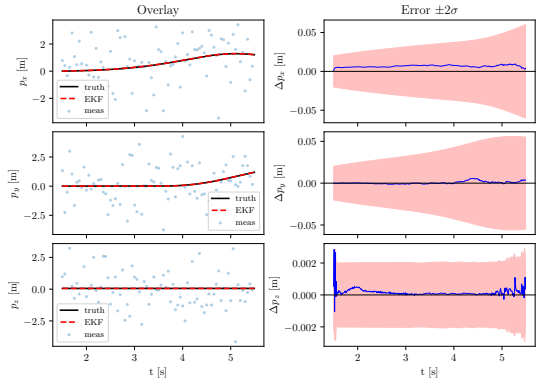

Fig. 1. Position data and 2σ errors for first test case. Maximum error in x and y was 4 and 6 cm, respectively.

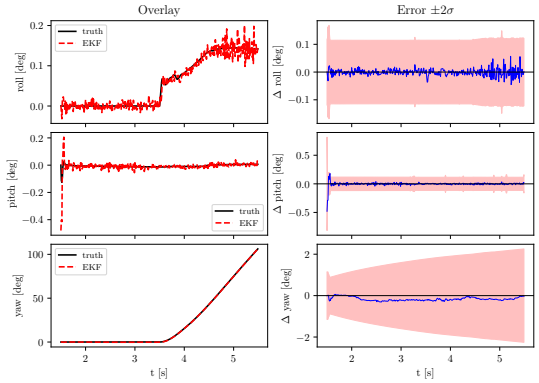

Fig. 2. Orientation data and 2σ errors for first test case. Maximum error in yaw is $\approx 1°$, while the other angles had error bounds on the order of $0.1°$ respectively.

B. Sinusoidal Trajectory

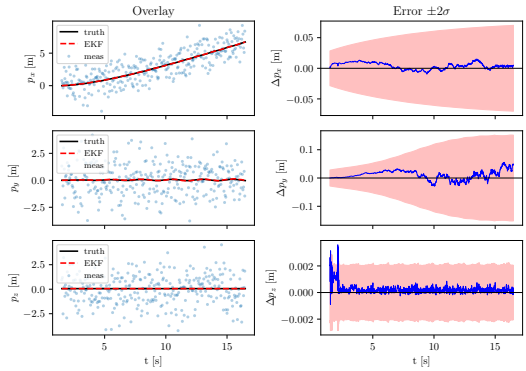

Fig. 3. Position data and 2σ errors for second test case. Maximum error in x and y was 5 and 10 cm, respectively.

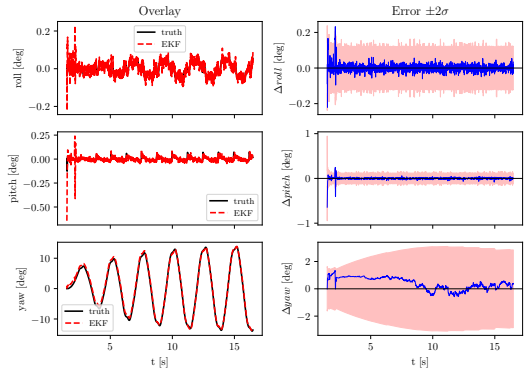

Fig. 4. Orientation data and 2σ errors for second test case. Maximum error in yaw is $\approx 2°$, while the other angles had error bounds on the order of $0.1°$ and $1°$ respectively.

TABLE I
EKF ERRORS FOR TEST CASES & SELECTED PARAMETERS

Parameter	**Position** (m)			**Velocity** (m/s)			**Yaw** (rad)		
Test	RMSE	Max	Min	RMSE	Max	Min	RMSE	Max	Min
Straight & Left Turn	0.007	0.009	0.001	0.035	0.34	0.001	0.064	0.064	0.064
Sinusoidal	0.020	0.055	0.001	0.035	0.301	0.001	0.405	0.405	0.405

V. REFLECTION

From the results of the test cases, the filter worked quite well for the given system parameters. For both tests, the maximum error did not exceed **5 cm**, while also maintaining yaw accuracy to within **0.5 rad**. Furthermore, while not shown in the figures, the velocity accuracy for both tests was within **0.35 m/s** maximum. Along with the incredible accuracy from both filters, the code is extremely efficient. On an Apple M2 Pro as a test bench, both test cases ran at **0.5 ms/predict *and* update step**, over a period of 1500 iterations. As this framework was formulated specifically to run on less capable, miniature hardware such as an NVIDIA Jetson, this combination of accuracy and speed lends itself extremely well to a power-efficient localization application. Also, from all figures we notice that the $\pm 2\sigma$ error bounds for each parameter converges for all parameters - this is extremely important in the health of the filter. While the bounds for stationary parameters such as roll and pitch are a bit noisy, the bounds for yaw, and position in both test cases are quite smooth. This is a large indication that the orientation update and body kinematics are working in the way that we expect.

However, the current framework is not without its drawbacks. The main problem with the currrent dynamics formulation is that even with the two separate wheel ODEs from equations (6) and (7), at slow RPMs the undriven wheels consistently switch directions, even though the friction coefficient μ is quite high, and the car is very unlikely to slip in the modeled conditions. Additionally, the oscillations shown in Figures 2 and 4 for the roll and pitch angles are likely due to the fact that the dynamics do not account for suspension, so the car is essentially a rigid body with no compliance, which is not entirely accurate. Furthermore, the current wheel dynamics are quite high frequency as well, with the current eigenvalues of the system exceeding 12. This means that for traditional forward Euler, a timestep of *1.67 μs* is required to accurately resolve the dynamics, which is orders of magnitude too quick for accurate simulation.

Future improvements on this project will focus on adding additional integration methods to ensure stability for a wider range of Δt (i.e., backward Euler), while also further investigating the modeling of the wheels to find the root cause of negative RPMs while driving forward in nominal conditions.